

Contenido

1. Resumen Ejecutivo	3
2. Vulnerabilidades Criticas	3
2.1 Secreto JWT Hardcodeado con Fallback Predecible	3
2.2 Credenciales de Administrador Expuestas en elCodigo Fuente	4
2.3 Rate Limiting en Memoria (Se Pierde al Reiniciar)	4
2.4 Verificacion de Sesion sin Re-validacion de Rol	4
2.5 Logout No Invalida el Token JWT	5
2.6 Variable JWT_SECRET Ausente del Archivo .env	5
2.7 Endpoint DELETE /api/perfumes sin Confirmacion ni Proteccion	5
2.8 Endpoint de Seed Accesible en Primer Arranque	6
2.9 Ausencia de Limites de Longitud en Inputs de API	6
3. Advertencias de Nivel Medio	6
3.1 Registro con Email: Riesgo de Privacidad Innecesario	6
3.2 Cookie Secure Flag Inconsistente	7
3.3 Proteccion CSRF No Aplicada Uniformemente	7
3.4 SQLite: Limitaciones de Concurrencia y Persistencia	7
3.5 Archivos Duplicados de Rate Limiting	8
4. Persistencia de Datos 24/7	8
4.1 Estado Actual: Como Se Persisten los Datos	8
4.2 Por Que Las Cookies No Son Suficientes para Favoritos	8
4.3 Estrategia de Backup Recomendada	9

5. Propuesta: Autenticacion Sin Email	9
5.1 Modelo Propuesto: Username + Contraseña	9
5.2 Flujo de Registro Simplificado	9
5.3 Beneficios de Seguridad y Privacidad	10
6. Plan de Seguridad Reforzado para el Modo Administrador	10
6.1 Autenticacion de Dos Factores para Administradores	10
6.2 Proteccion contra Elevacion de Privilegios	10
6.3 Log de Auditoria	11
7. Buenas Practicas Ya Implementadas	11
8. Plan de Implementacion Priorizado	12

1. Resumen Ejecutivo

Este informe presenta un analisis exhaustivo de la seguridad y persistencia de datos del sitio web de Jolie Fragrances, construido con Next.js 16, TypeScript, Prisma ORM y SQLite. Tras revisar cada archivo del proyecto, se identificaron 9 vulnerabilidades criticas, 5 advertencias de nivel medio y 8 mejoras recomendadas que, de implementarse, elevarian la seguridad del sitio a un nivel profesional y robusto. Las vulnerabilidades mas graves incluyen un secreto JWT hardcodeado con valor fallback predecible, credenciales de administrador expuestas en el codigo fuente del endpoint de seed, rate limiting almacenado en memoria (se pierde al reiniciar el servidor), y ausencia de validacion de rol administrador en la verificacion de sesion. Ademias, el sistema de autenticacion actualmente requiere email, un dato que no se utiliza para ningun proposito operativo y que representa un riesgo de privacidad innecesario para los clientes.

Categoria	Cantidad	Severidad
Vulnerabilidades Criticas	9	CRITICA
Advertencias Medias	5	MEDIA
Mejoras Recomendadas	8	INFO
Buenas Practicas ya Implementadas	7	OK

Tabla 1: Resumen de hallazgos de seguridad

2. Vulnerabilidades Criticas

2.1 Secreto JWT Hardcodeado con Fallback Predecible

El archivo `src/lib/auth.ts` contiene la siguiente linea: `const JWT_SECRET = process.env.JWT_SECRET || 'jolie-fragrances-secret-key-2024-fallback'`. Este patron es extremadamente peligroso por dos razones complementarias. En primer lugar, el valor fallback esta expuesto directamente en el codigo fuente del repositorio, lo que significa que cualquier persona con acceso al codigo (incluyendo el repositorio Git) puede firmar tokens JWT validos y hacerse pasar por cualquier usuario, incluyendo administradores. En segundo lugar, la variable de entorno `JWT_SECRET` no esta definida en el archivo `.env`, lo que significa que en la practica el servidor SIEMPRE utiliza el fallback predecible.

El impacto es total: un atacante que conozca el secreto puede crear tokens JWT con `role: "admin"` y obtener acceso completo al panel de administracion, modificar perfumes, eliminar datos, y comprometer la integridad completa del sitio. La solucion inmediata es generar un secreto aleatorio de al menos 64 caracteres y almacenarlo exclusivamente en la variable de

entorno, eliminando completamente el fallback.

CRITICO - Un atacante puede forjar tokens de administrador y tomar control total del sitio.

2.2 Credenciales de Administrador Expuestas en el Código Fuente

El endpoint `/api/seed` contiene las credenciales del administrador hardcodeadas: `const adminEmail = "jolie@fragrances.com"; const adminPassword = "Jolie$Frag2024!Secure";`. Aunque este endpoint solo se ejecuta una vez para inicializar la base de datos, el hecho de que las credenciales estén en el código fuente es una violación grave de seguridad. Si el repositorio es público o si alguien obtiene acceso al código, las credenciales administrativas quedan completamente expuestas. Además, la contraseña no se rota automáticamente, lo que significa que si se descubre, no hay un mecanismo para cambiarla sin intervenir manualmente en la base de datos.

La solución recomendada es que el endpoint de seed lea las credenciales administrativas desde variables de entorno, y que se implemente un flujo de "primer arranque" donde la contraseña se genere aleatoriamente y se muestre una única vez en los logs del servidor, obligando al administrador a cambiarla en el primer inicio de sesión.

CRITICO - Credenciales admin visibles en el código fuente del repositorio.

2.3 Rate Limiting en Memoria (Se Pierde al Reiniciar)

El proyecto tiene DOS implementaciones de rate limiting (`src/lib/rate-limit.ts` y `src/lib/security.ts`), ambas basadas en Map en memoria. Esto significa que cada vez que el servidor se reinicia (lo cual ocurre automáticamente en la mayoría de plataformas de despliegue), todos los contadores de rate limiting se reinician a cero. Un atacante puede aprovechar esto simplemente enviando solicitudes masivas inmediatamente después de un reinicio del servidor, o incluso provocando el reinicio deliberadamente para resetear los contadores.

Además, en un entorno con múltiples instancias del servidor (escalado horizontal), cada instancia mantiene su propio Map de rate limiting, lo que efectivamente multiplica el límite permitido por el número de instancias. Si el límite es de 5 intentos de login por IP, y hay 3 instancias, un atacante tiene 15 intentos. La solución es mover el rate limiting a la base de datos (tabla dedicada) o a un almacén persistente como Redis, de modo que los contadores sobrevivan reinicios y sean consistentes entre instancias.

CRITICO - Protección contra fuerza bruta ineficaz tras reinicio del servidor.

2.4 Verificación de Sesión sin Re-validación de Rol

El endpoint `/api/auth/session` verifica el token JWT y luego consulta la base de datos para obtener los datos actualizados del usuario. Sin embargo, la verificación de favoritos y otras operaciones críticas solo validan que el token sea válido, sin re-verificar el rol del usuario contra

la base de datos. El flujo es: el token JWT contiene el rol del usuario en el momento de la emision, y la funcion `verifyToken()` solo verifica la firma del token, no si el rol sigue siendo valido.

El escenario de ataque es el siguiente: si un administrador es degradado a usuario normal, su token JWT anterior (que contiene `role: "admin"`) seguira siendo valido durante los 7 dias de vida del token, permitiendole realizar acciones administrativas. La solucion es que cada endpoint administrativo verifique el rol del usuario directamente desde la base de datos, no desde el token JWT. Alternativamente, se puede implementar una lista de tokens revocados o reducir significativamente la vida del token.

CRITICO - Admin degradado mantiene acceso por hasta 7 dias.

2.5 Logout No Invalida el Token JWT

El endpoint `/api/auth/logout` simplemente elimina la cookie del navegador, pero NO invalida el token JWT en el servidor. Esto significa que si un atacante obtiene el token (por ejemplo, mediante XSS o interceptacion de red), puede seguir usandolo incluso despues de que el usuario cierre sesion. El token permanece valido hasta su expiracion natural (7 dias). La cookie se establece con `secure: false` en el logout, lo que tambien es inconsistente con la configuracion de login que usa `secure: process.env.NODE_ENV === "production"`.

La solucion requiere implementar un mecanismo de revocacion de tokens, ya sea mediante una tabla de tokens revocados en la base de datos, o mediante una lista negra almacenada en cache. Cada vez que se cierra sesion, el token se agrega a la lista negra, y la funcion `verifyToken()` verifica que el token no este revocado antes de aceptarlo.

CRITICO - Token JWT sigue activo despues del logout.

2.6 Variable JWT_SECRET Ausente del Archivo .env

El archivo `.env` solo contiene `DATABASE_URL=file:/home/z/my-project/db/custom.db`. No existe ninguna variable `JWT_SECRET` definida, lo que confirma que el servidor utiliza el fallback hardcodeado. Ademas, no hay variables de entorno para otras configuraciones sensibles como la URL de la base de datos de produccion, claves API, o el modo de entorno. El archivo `.gitignore` si excluye `.env`, lo cual es correcto, pero la ausencia de un archivo `.env.example` hace que los desarrolladores no sepan que variables necesitan configurar.

CRITICO - El secreto JWT nunca se carga desde entorno.

2.7 Endpoint DELETE /api/perfumes sin Confirmacion ni Proteccion

El endpoint DELETE /api/perfumes permite eliminar TODOS los perfumes de la base de datos cuando no se proporciona un ID. Esto es extremadamente peligroso porque un solo request puede borrar todo el catalogo de la tienda. Aunque requiere autenticacion de administrador, combinado con las vulnerabilidades de falsificacion de tokens JWT (secciones 2.1 y 2.4), un atacante podria eliminar todo el inventario con un simple comando curl. La solucion es eliminar la funcionalidad de "borrar todo" o requerir una confirmacion de dos pasos, como enviar un codigo de confirmacion que se muestre en el panel admin y deba ser reenviado.

CRITICO - Un solo request puede eliminar todo el catalogo.

2.8 Endpoint de Seed Accesible en Primer Arranque

El endpoint POST /api/seed tiene una logica que permite la ejecucion sin autenticacion cuando la base de datos esta vacia (`if (userCount > 0) {...check auth...}`). Si por cualquier motivo la base de datos se reinicia o se corrompe, cualquier persona puede ejecutar el seed y crear la cuenta de administrador con las credenciales hardcodedas. Esto podria permitir que un atacante tome control del sitio antes de que el administrador legitimo pueda reconfigurar las credenciales.

CRITICO - Seed sin auth cuando la DB esta vacia.

2.9 Ausencia de Limites de Longitud en Inputs de API

Los endpoints de la API no imponen limites estrictos en la longitud de los campos de entrada. El registro permite contraseñas de longitud ilimitada (el hash bcrypt tiene un limite interno de 72 bytes, pero el procesamiento de la solicitud en si no esta limitado), y los campos como name solo se truncan a 100 caracteres sin validacion de formato. Los endpoints de perfumes (POST/PUT) no tienen limites en los campos de texto como name, brand, o perfumeSlug. Esto puede permitir ataques de denegacion de servicio enviando payloads extremadamente grandes, o inyecciones de datos maliciosos.

CRITICO - Sin limites de payload, vulnerable a DoS.

3. Advertencias de Nivel Medio

3.1 Registro con Email: Riesgo de Privacidad Innecesario

Actualmente el sistema requiere un email para registrarse, pero este dato no se utiliza para ningun proposito operativo: no se envian correos de bienvenida, no hay recuperacion de contraseña por email, no se usa para marketing, y no se envian notificaciones. Esto significa que estas almacenando datos personales sensibles (emails de clientes) sin ninguna justificacion funcional, exponiendo a los usuarios a un riesgo innecesario en caso de brecha de seguridad. Si la base de

datos es comprometida, los emails de todos los clientes quedarian expuestos, podrian ser vendidos a terceros o utilizados para ataques de phishing dirigidos.

Tu consejero anterior tenia razon: eliminar el email del registro es una mejora significativa de privacidad. La alternativa es usar un nombre de usuario unico (username) como identificador principal, lo que ademas es mas intuitivo para los clientes que no quieren compartir su email personal. El email se puede mantener como campo opcional para futuras funcionalidades, pero nunca como requisito obligatorio.

MEDIO - Datos personales almacenados sin justificacion funcional.

3.2 Cookie Secure Flag Inconsistente

En el login, la cookie se establece con `secure: process.env.NODE_ENV === "production"`, pero en el logout se usa `secure: false`. Esta inconsistencia puede causar que la cookie de logout no elimine correctamente la cookie de login en produccion (si la cookie original se establecio con `secure:true`, el logout con `secure:false` no la eliminara en navegadores que exigen coincidencia de flags). Ademas, la cookie usa `sameSite: "lax"`, lo cual es correcto para la mayoria de los casos, pero deberia ser `sameSite: "strict"` para endpoints de autenticacion que no necesitan ser accedidos desde sitios externos.

MEDIO - Cookie de logout puede fallar en produccion.

3.3 Proteccion CSRF No Aplicada Uniformemente

El archivo `src/lib/csrf.ts` implementa una verificacion de origen valida, pero revisando los endpoints de la API, ninguno importa o utiliza la funcion `verifyOrigin()`. El archivo existe pero no se aplica en ningun lado. Esto significa que las solicitudes POST, PUT y DELETE pueden ser realizadas desde sitios web de terceros si el usuario tiene una sesion activa. La funcion `addSecurityHeaders()` del archivo `security-headers.ts` tampoco se aplica en ningun endpoint.

MEDIO - Proteccion CSRF implementada pero no aplicada.

3.4 SQLite: Limitaciones de Concurrencia y Persistencia

SQLite es excelente para desarrollo y proyectos pequenos, pero tiene limitaciones importantes para un sitio web en produccion que necesita estar disponible 24/7. Primero, SQLite solo permite un escritor a la vez, lo que significa que si dos usuarios intentan agregar un favorito simultaneamente, uno tendra que esperar. Segundo, la base de datos es un archivo en disco; si el servidor se reinicia o se migra, el archivo debe estar en la ubicacion exacta especificada en `DATABASE_URL`. Tercero, no hay mecanismo de backup automatico, por lo que si el archivo se corrompe, todos los datos se pierden (favoritos, usuarios, perfumes personalizados).

Para un emprendimiento que necesita disponibilidad continua, se recomienda migrar a PostgreSQL cuando el trafico supere los 100 usuarios concurrentes. Mientras tanto, es imprescindible implementar backups automaticos del archivo SQLite (al menos diarios) y almacenarlos en una ubicacion externa.

MEDIO - Riesgo de perdida de datos sin backups.

3.5 Archivos Duplicados de Rate Limiting

Existen dos archivos que implementan rate limiting con interfaces diferentes: `src/lib/rate-limit.ts` y `src/lib/security.ts`. Ambos usan Map en memoria, pero con APIs incompatibles. El archivo `rate-limit.ts` usa `checkRateLimit(id, config)` que retorna `{limited, retryAfterMs}`, mientras que `security.ts` usa `checkRateLimit(id, options)` que retorna `{allowed, retryAfter}`. El endpoint de login usa el de `security.ts`, pero no esta claro cual deberia usarse para nuevos endpoints. Esto genera confusion y posibles inconsistencias.

MEDIO - Dos implementaciones incompatibles del mismo concepto.

4. Persistencia de Datos 24/7

4.1 Estado Actual: Como Se Persisten los Datos

El sitio utiliza Prisma ORM con SQLite para almacenar usuarios, perfumes y favoritos. Los favoritos se guardan en la tabla Favorite de la base de datos, vinculados al ID del usuario. Esto significa que los favoritos SI persisten entre sesiones y entre reinicios del servidor, siempre que el archivo de base de datos no se pierda. Sin embargo, hay un punto critico: la ruta de la base de datos en el archivo `.env` apunta a `file:/home/z/my-project/db/custom.db`, que es una ruta absoluta del sistema de archivos. Si el servidor se migra o se reinstala, esta ruta debe actualizarse o los datos se perderan.

4.2 Por Que Las Cookies No Son Suficientes para Favoritos

Tu idea de usar cookies para favoritos tiene logica intuitiva, pero tiene limitaciones importantes. Las cookies tienen un tamano maximo de 4KB por cookie y 20 cookies por dominio en algunos navegadores. Si un usuario guarda 50 perfumes como favoritos (cada uno con ID, nombre y marca), se supera facilmente este limite. Ademas, las cookies se envian en CADA solicitud HTTP, incluyendo imagenes y archivos estaticos, lo que degrada el rendimiento. Finalmente, las cookies son vulnerables a manipulacion por parte del usuario y pueden ser eliminadas accidentalmente al limpiar el cache del navegador.

La solucion correcta es la que ya tienes implementada: favoritos en la base de datos vinculados al usuario. Lo que necesitas mejorar es la experiencia para usuarios no registrados. Puedes usar `localStorage` del navegador para almacenar favoritos temporales de usuarios no autenticados, y cuando se registren, migrar esos favoritos a la base de datos. Esto te da lo mejor de ambos mundos: favoritos inmediatos sin registro, y persistencia real con registro.

4.3 Estrategia de Backup Recomendada

Para garantizar que los datos persistan 24/7, necesitas un sistema de backup automatico. La estrategia minima consiste en tres componentes: un backup diario completo del archivo SQLite almacenado en una ubicacion externa (como un servicio de almacenamiento en la nube), un mecanismo de rotacion que mantenga los ultimos 7 backups diarios y 4 backups semanales, y un script de restauracion que pueda recuperar la base de datos desde un backup en menos de 5 minutos. Para un nivel superior de proteccion, se recomienda replicar la base de datos en tiempo real a un servidor secundario, de modo que si el servidor principal falla, el secundario puede tomar el control sin perdida de datos.

5. Propuesta: Autenticacion Sin Email

5.1 Modelo Propuesto: Username + Contraseña

Se propone reemplazar el email como campo obligatorio de registro por un nombre de usuario (username) unico. El email pasaria a ser un campo completamente opcional, nunca requerido, nunca verificado, y solo almacenado si el usuario decide proporcionarlo voluntariamente. El modelo de datos quedaria asi:

Campo	Tipo	Requerido	Unico
username	String	Si	Si
password	String (hash)	Si	No
email	String?	No	Si (si se provee)
name	String?	No	No
role	String	Default: user	No

Tabla 2: Modelo de datos propuesto para usuarios

5.2 Flujo de Registro Simplificado

El nuevo flujo de registro consistiría en solo dos campos obligatorios: nombre de usuario y contraseña. Opcionalmente, el usuario puede agregar su nombre real y/o email, pero ninguno de estos campos sería necesario para crear la cuenta. Esto reduce la fricción de registro y aumenta la confianza del usuario al no pedir datos personales innecesarios. El nombre de usuario debe tener reglas claras: entre 3 y 30 caracteres, solo letras, números y guiones bajos, sin espacios ni caracteres especiales. La contraseña debe mantener los requisitos actuales de complejidad (8+ caracteres, mayúscula, número, especial).

5.3 Beneficios de Seguridad y Privacidad

Eliminar el email obligatorio tiene múltiples beneficios tangibles. Primero, reduces la superficie de ataque: si la base de datos es comprometida, no hay emails que puedan ser utilizados para phishing o vendidos a terceros. Segundo, los clientes se sienten más seguros al no tener que proporcionar datos personales sensibles para probar la plataforma. Tercero, simplificas el compliance con regulaciones de protección de datos (como la LOPD en Venezuela o el GDPR si expands a otros mercados), ya que no almacenas datos personales innecesarios. Cuarto, reduces la posibilidad de ataques de enumeración de usuarios mediante el endpoint de registro (actualmente devuelve "Este email ya está registrado", lo que permite a un atacante descubrir que emails están en la base de datos).

6. Plan de Seguridad Reforzado para el Modo Administrador

6.1 Autenticación de Dos Factores para Administradores

La protección más efectiva contra el acceso no autorizado al panel de administración es la autenticación de dos factores (2FA). Dado que tu emprendimiento no necesita una solución empresarial compleja, la implementación más práctica es TOTP (Time-based One-Time Password), que es el mismo sistema que usan aplicaciones como Google Authenticator o Authy. El flujo sería: el admin ingresa username y contraseña, el servidor genera un secreto TOTP único, el admin escanea un código QR con su app de autenticación, y a partir de ese momento necesita el código de 6 dígitos además de la contraseña. Esto hace que incluso si la contraseña es comprometida, el atacante no puede acceder sin el dispositivo físico del administrador.

6.2 Protección contra Elevación de Privilegios

Para evitar que un usuario normal pueda elevar sus privilegios a administrador, necesitas tres capas de protección. Primero, el campo role de la base de datos nunca debe ser modificable desde la API de registro o actualización de perfil. Segundo, los endpoints administrativos deben

verificar el rol directamente desde la base de datos en CADA solicitud, no desde el token JWT. Tercero, los cambios de rol deben requerir la autenticacion de un administrador existente mas 2FA, y deben quedar registrados en un log de auditoria que incluya quien hizo el cambio, cuando, y a quien se le otorgo o revoco el rol.

6.3 Log de Auditoria

Un log de auditoria es fundamental para detectar y responder a incidentes de seguridad. El modelo propuesto registraria cada accion sensible: inicios de sesion (exitosos y fallidos), cambios de rol, creacion y eliminacion de perfumes, modificaciones de datos, y accesos al panel de administracion. Cada entrada incluiria: ID del usuario, accion realizada, recurso afectado, timestamp, direccion IP, y agente de usuario. Este log se almacenaria en una tabla de solo adicion (no se puede modificar ni eliminar), y serviria como evidencia forense en caso de incidente.

Campo	Tipo	Descripcion
id	String (cuid)	Identificador unico
userId	String	Usuario que realizo la accion
action	String	Tipo de accion (login, create, delete, etc.)
resource	String	Recurso afectado (perfume, user, etc.)
resourceId	String?	ID del recurso afectado
ipAddress	String	IP del cliente
userAgent	String?	Navegador/dispositivo del cliente
createdAt	DateTime	Momento de la accion

Tabla 3: Modelo de datos para log de auditoria

7. Buenas Practicas Ya Implementadas

Es importante reconocer que el proyecto ya tiene varias medidas de seguridad correctamente implementadas que forman una base solida. Estas buenas practicas deben mantenerse y complementarse con las mejoras propuestas.

Practica	Implementacion	Estado
Hashing de contraseñas	bcrypt con salt rounds 12	OK
Cookies httpOnly	Token JWT en cookie httpOnly	OK
SameSite cookie	sameSite: lax en cookies	OK

Rate limiting basico	5 intentos login / 15 min por IP	OK
Sanitizacion de inputs	sanitizeString() elimina HTML/injection	OK
Validacion de contrasena	8+ chars, mayuscula, numero, especial	OK
Verificacion de admin en DB	Admin endpoints check role en DB	OK

Tabla 4: Buenas practicas de seguridad ya implementadas

8. Plan de Implementacion Priorizado

Las siguientes mejoras estan ordenadas por prioridad de implementacion, considerando tanto el impacto en seguridad como la complejidad de implementacion. Las primeras tres son criticas y deben implementarse antes de poner el sitio en produccion publica.

Prioridad	Mejora	Severidad	Esfuerzo
1	JWT_SECRET desde variable de entorno (sin fallback)	CRITICA	1 hora
2	Credenciales admin desde variables de entorno	CRITICA	2 horas
3	Aplicar verifyOrigin() y addSecurityHeaders() en todos los endpoints	CRITICA	3 horas
4	Revocacion de tokens JWT en logout	CRITICA	4 horas
5	Re-verificacion de rol desde DB en endpoints admin	CRITICA	2 horas
6	Rate limiting en base de datos (persistente)	CRITICA	4 horas
7	Eliminar email obligatorio, usar username	MEDIA	6 horas
8	Backup automatico de SQLite	MEDIA	3 horas
9	Favoritos temporales con localStorage + migracion	MEDIA	4 horas
10	2FA TOTP para administradores	INFO	8 horas
11	Log de auditoria	INFO	6 horas
12	Limites de longitud en todos los inputs de API	CRITICA	3 horas
13	Eliminar DELETE masivo de perfumes o agregar confirmacion	CRITICA	1 hora
14	Corregir cookie flags en logout	MEDIA	15 min

Tabla 5: Plan de implementacion priorizado

